Resources Used

Socket.IO Documentation (socket.io/docs/v4)

Used to implement real-time communication between the server and all connected clients. Learned how to emit events to a specific room, handle connections and disconnections, and send state updates to all participants simultaneously.

~2–3 hours+

LiveKit JavaScript Client SDK (docs.livekit.io)

Used to add video and audio between participants. Learned how to connect to a room using a token, publish local camera and microphone tracks, and subscribe to remote participants' video feeds using the trackSubscribed event.

~3–4 hours

LiveKit Server SDK for Node.js (docs.livekit.io) Used to generate secure access tokens on the backend. Learned how to create a signed JWT with a room grant so only authenticated users can join a video room. Spent about 1 hour on this.

MDN — MediaDevices.getUserMedia()

Used to request camera and microphone access from the browser before joining. Learned how the permission prompt works and how to extract video and audio tracks from the returned stream.

~30 minutes

MDN — Web Audio API and AnalyserNode

Used to detect who is speaking at any moment. Learned how to build an audio processing graph, connect a media stream to an analyser node, and read raw waveform data using getByteTimeDomainData().

~1–2 hrs

Stack Overflow — RMS volume calculation (stackoverflow.com/questions/7854236)

Used to figure out the correct formula for measuring audio loudness from the analyzer output. Learned that RMS reflects perceived volume better than peak values.

~30 minutes

Clipboard API (navigator.clipboard.writeText)

Used for the Copy Invite button. Learned that it returns a Promise and requires a fallback for browsers that block clipboard access.

~1 hr.

MDN — CSS Grid Layout and CSS-Tricks Complete Guide to Grid Used to build the dynamic video grid that reconfigures as participants join or leave. Learned how repeat (), 1fr, and grid-template-columns work together to fill available space evenly.

1–2 hours

Architecture:

- **Browser** — three modules working together: Socket.IO client (state/events), LiveKit client (video/audio), Web Audio API (who is speaking). All three are fed by getUserMedia() at the bottom.
- **Node.js Server** — two parts: Socket.IO handles all room logic, timer, VAD reports, and host enforcement. Express serves the page and signs LiveKit tokens via the /get-token endpoint.
- **LiveKit Cloud** — the external WebRTC media server that routes video and audio between participants. It never touches your app logic — it only receives the JWT the server signed and forwards media.
- **Arrows** show the three separate channels: WebSocket for state, HTTP for the token, and WebRTC for the actual video/audio.